

C-GEAR: Calibrated Genetic Efficiency-Aware Rank Allocation for Incremental LoRA Fine-Tuning

Ali Naghiloo (404200139) Mohammad Farid Barenji (404203169)

Sharif University of Technology, Department of Electrical Engineering

Deep Learning — Instructor: Dr. Bejani; Mentor: Eng. Mehran Advand

Abstract

Incremental LoRA avoids fixing a large adapter rank in advance, but IncreLoRA assigns each new component through a local importance-based Greedy rule and continues growing toward a preset target. We introduce *C-GEAR*, which anchors evolutionary candidate generation at that rule, calibrates candidate updates on training data only, selects a budget-feasible variable event size (including no growth), and stops allocation when positive growth is repeatedly unreliable. On GLUE RTE with DeBERTa-v3-base and six matched seeds, C-GEAR improves mean selected-checkpoint accuracy from 87.30% to 88.09% while reducing selected active parameters by 3.55% on average per matched pair. Terminal allocation trajectories use 13.86% fewer active parameters. These descriptive results establish a favorable local accuracy–efficiency trade-off without claiming broad or statistically significant superiority.

1 Introduction

Low-rank adaptation (LoRA) freezes a pretrained model and learns low-rank updates, sharply reducing task-specific storage [3]. Yet a uniform fixed rank ignores that attention and feed-forward matrices need not contribute equally. IncreLoRA instead begins small and allocates rank during optimization [7], but its locally Greedy allocator asks only which modules currently look most important; it cannot compare coordinated alternatives or decline an allocation event.

This work asks whether incremental rank growth can become more selective and parameter-efficient without sacrificing downstream performance. C-GEAR answers with (i) Greedy-anchored evolutionary search over non-uniform module sets, (ii) conservative training-only calibration, (iii) adaptive event size k including $k = 0$, and (iv) explicit budget and stopping logic. Beyond the allocator, the project contributes dynamic-checkpoint reconstruction, candidate snapshot/restore, budget-correct parameter accounting, regression tests, and a schema-validated telemetry/analysis pipeline. Code and reproducibility artifacts are available at <https://github.com/Aliflori/colabLoRA>.

Figures 1 and 2 preview the event-level decision flow and the search that supplies its candidate allocations; Section 3 specifies the implementation.

2 Related Work

Adaptive LoRA methods relax a single hand-chosen rank in different ways. AdaLoRA allocates a global budget by importance [8]; DyLoRA jointly trains a nested range of usable ranks [5]; SoRA learns sparse rank gates [1]; AutoLoRA uses meta-learning to tune layer ranks [9]; and dynamic-rank DoRA prunes rank-one components [4].

(a) C-GEAR allocation event

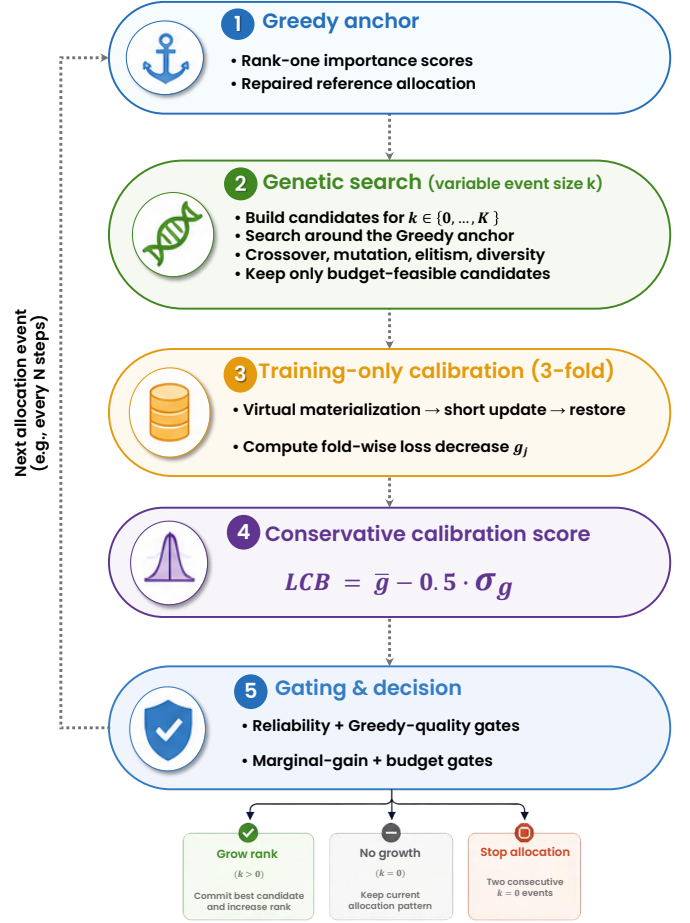


Figure 1. Overall C-GEAR allocation-event flow. Genetic search proposes feasible module sets; training-only calibration and conservative gates choose growth, no growth ($k = 0$), or stopping after repeated no-growth events.

IncreLoRA reverses pruning by progressively adding components, avoiding a restrictive initial upper rank [7]. C-GEAR retains this incremental setting but targets a different gap: *which combination* should grow at an event, by how much, and whether growth is warranted at all.

3 Method / Proposed Approach

Let module i have active rank r_i and dimensions $d_{in,i}$ and $d_{out,i}$. Its active adaptation cost is

$$C(\mathbf{r}) = \sum_i r_i (d_{in,i} + d_{out,i}), \quad C(\mathbf{r}) \leq B. \quad (1)$$

At each event, C-GEAR first proposes budget-feasible allocations, then uses training-only evidence to decide among growth, no growth, and stopping (Figure 1).

(b) Inside the genetic search — example $k = 3$

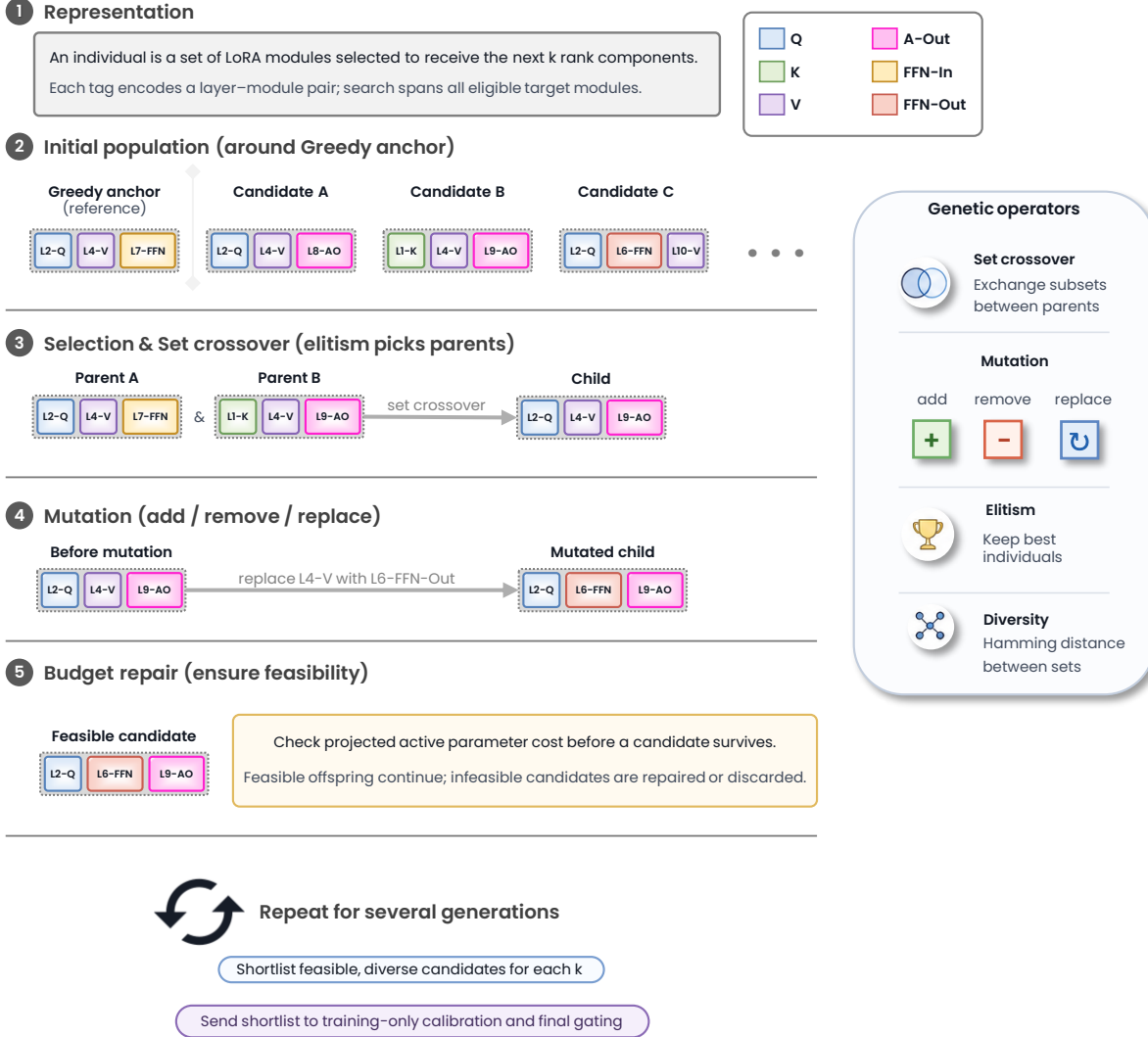


Figure 2. Inside the genetic search: an illustrative $k = 3$ example, not a trace of a particular event. Individuals encode sets of layer-module pairs; Greedy anchoring, set crossover, add/remove/replace mutation, budget repair, elitism, and diversity produce a feasible calibration shortlist.

3.1 Genetic Rank-Allocation Search

An individual is the set of LoRA modules receiving the next k rank-one components (Figure 2). Legal sizes are $k \in \{0, \dots, K\}$, with the empty set representing a genuine no-growth action. The original IncreLoRA Greedy allocation remains the anchor and quality reference: C-GEAR retains a budget-repaired anchor and Greedy prefixes, enumerates nearby replacements, and seeds global populations with the anchor, cost/gain extremes, and random sets. Structural fitness combines normalized importance, complementary temporal interactions, redundancy, and relative parameter cost.

Tournament-selected parents undergo set crossover; mutation can add, remove, or replace modules. Deterministic repair substitutes cheaper modules or reduces cardinality until projected active cost satisfies B . Environmental selection retains a strong elite, then applies maximin normalized set-Hamming pressure, where membership-vector distance is $|A \triangle B| / \max(|A|, |B|, 1)$, to discourage population collapse. After the configured generations, representative feasi-

ble candidates form the shortlist. No post-hoc local search is used. Evolution therefore proposes allocations from training-derived signals; it does not optimize against RTE validation labels.

3.2 Training-Only Calibration and Commit Decision

Search alone cannot commit a change. Each shortlist candidate is virtually materialized, updated on three held-out *training* folds, and restored with model, optimizer, rank, and random-number state unchanged. For loss decrease g_j on fold j , the implemented conservative score is

$$S_{\text{cal}} = \bar{g} - \lambda \sigma_g, \quad \lambda = 0.5. \quad (2)$$

Here $\bar{g}$ rewards expected improvement, while σ_g penalizes instability across folds. The score is a conservative LCB-style heuristic, not a formal 95% confidence bound. A positive candidate must clear validity, marginal-gain, Greedy-quality, and budget gates; global-only candidates face a stricter improvement gate. Within the quality band, projected active cost is minimized before calibrated quality breaks ties. If

Table 1. Six-seed RTE results (mean $\pm$ sample SD). Accuracy and selected counts describe the same selected checkpoint; final counts are architectural.

Metric	Greedy IncreLoRA	C-GEAR
Accuracy $\uparrow$	87.30 $\pm$ 0.84	88.09 $\pm$ 1.14
Selected params $\downarrow$	828.0k $\pm$ 65.2k	795.2k $\pm$ 37.1k
Selected rank $\downarrow$	99.0 $\pm$ 27.8	86.7 $\pm$ 14.5
Final params $\downarrow$	933.6k $\pm$ 5.3k	804.1k $\pm$ 39.3k
Final rank $\downarrow$	144.0 $\pm$ 0.0	89.7 $\pm$ 15.0

none qualifies, $k = 0$ preserves the pattern; two consecutive no-growth events stop allocation and begin ordinary consolidation training. Calibration uses training data only and never validation labels.

4 Experimental Setup

We perform a controlled local comparison on GLUE RTE [6] with `microsoft/deberta-v3-base` [2]. Greedy IncreLoRA and C-GEAR use identical seeds 41–46, data order, model, targets, and optimization: 25 epochs (1,950 steps), sequence length 192, train/evaluation batches 32/8, AdamW at 1.2×10^{-3} , linear scheduling with 100 warm-up steps, weight decay 0.01, and FP16 on an RTX 4060 Laptop GPU. Allocation begins from rank one in 72 target modules and is considered every 100 steps. C-GEAR uses population 12, four generations, calibration top-6, budget ratio 0.94, and the fixed settings recorded by telemetry. RTE validation accuracy is the performance metric.

Efficiency is *active model parameters*: the fixed task head plus components represented by the active rank pattern, not raw `requires_grad` storage. “Selected” denotes the checkpoint chosen by validation accuracy after dynamic-rank restoration; “final” denotes the terminal allocation trajectory before that checkpoint is reloaded. We pair accuracy only with selected-checkpoint counts. This is a matched local reproduction, not a direct numerical comparison with differently configured results in the original IncreLoRA paper.

5 Results & Contributions

5.1 Outcome-Level Evidence

Table 1 gives the central comparison. C-GEAR raises mean accuracy by 0.78 percentage points and wins/ties/loses 4/0/2 matched seeds. At those *same selected checkpoints*, mean active parameters fall from 828,005 to 795,225 and mean active rank from 99.0 to 86.7. The primary paired reduction is 3.55%; the reduction computed from aggregate mean counts is 3.96%, illustrating why the paired statistic must be labeled.

Figure 3 exposes rather than hides variation: C-GEAR loses on seeds 42 and 43, and selected efficiency is not lower for every pair. Nonetheless, the mean selected-checkpoint trade-off favors C-GEAR. Separately, final trajectories average 804,060 active parameters and rank 89.7, versus Greedy’s 933,650 and rank 144; the mean paired final-parameter reduction is 13.86%. These terminal counts do *not* inherit selected-checkpoint accuracy.

5.2 Mechanistic Analysis

Figure 4 separates rank-growth behavior from the outcome comparison. Greedy adds 72 ranks in every run and reaches rank 144. C-GEAR’s 55 genuine calibrated decisions use every event size from $k = 0$ to $k = 5$: 15 select no growth, while the positive decisions produce seed-dependent terminal

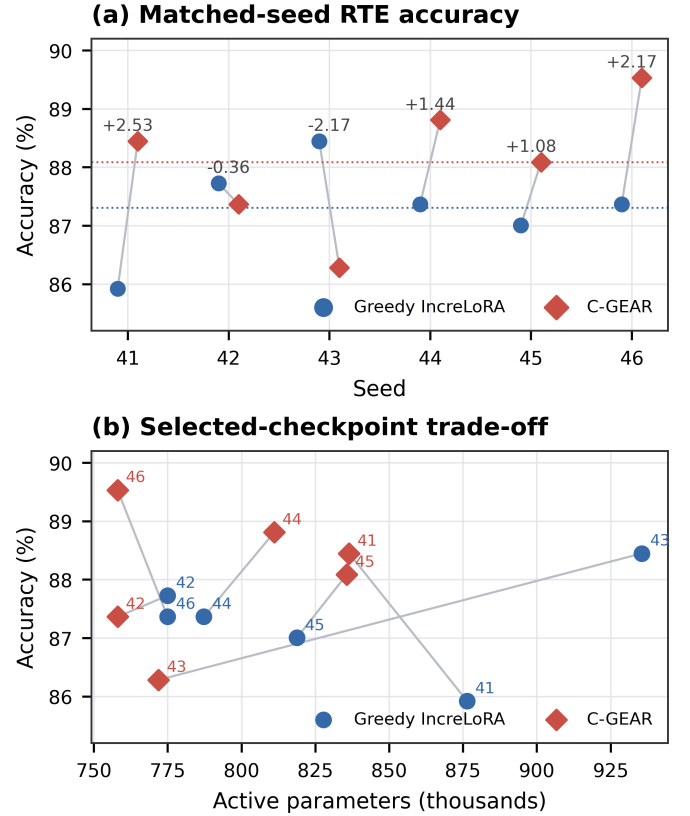


Figure 3. Matched results. (a) Per-seed accuracy and C-GEAR minus Greedy differences in percentage points. (b) Accuracy versus active parameters at the selected checkpoint; each gray segment joins a matched seed.

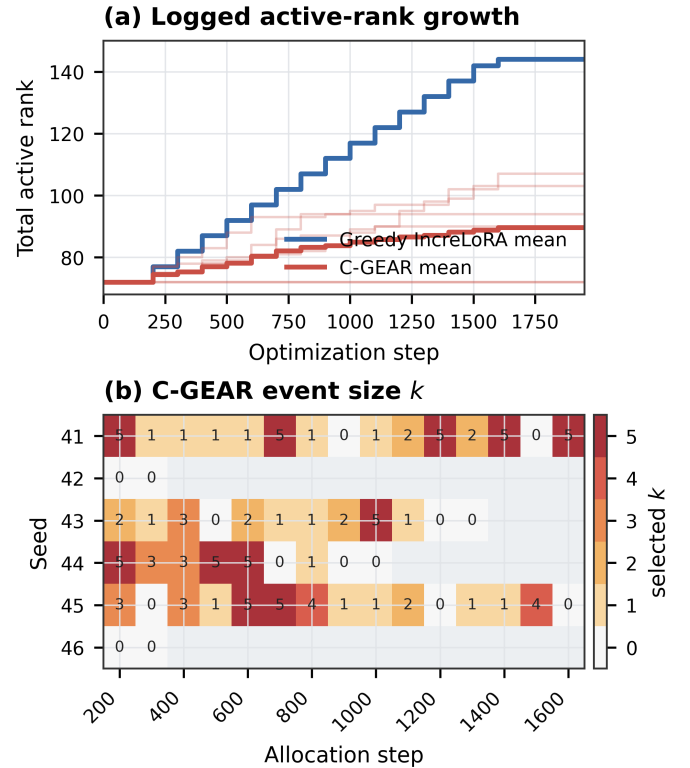


Figure 4. Telemetry-derived growth. Top: all six trajectories and method means. Bottom: C-GEAR’s selected event size; blank cells occur after allocation stops.

ranks from 72 to 107. Four runs stop after consecutive $k = 0$ decisions; two instead enter the required final consolidation

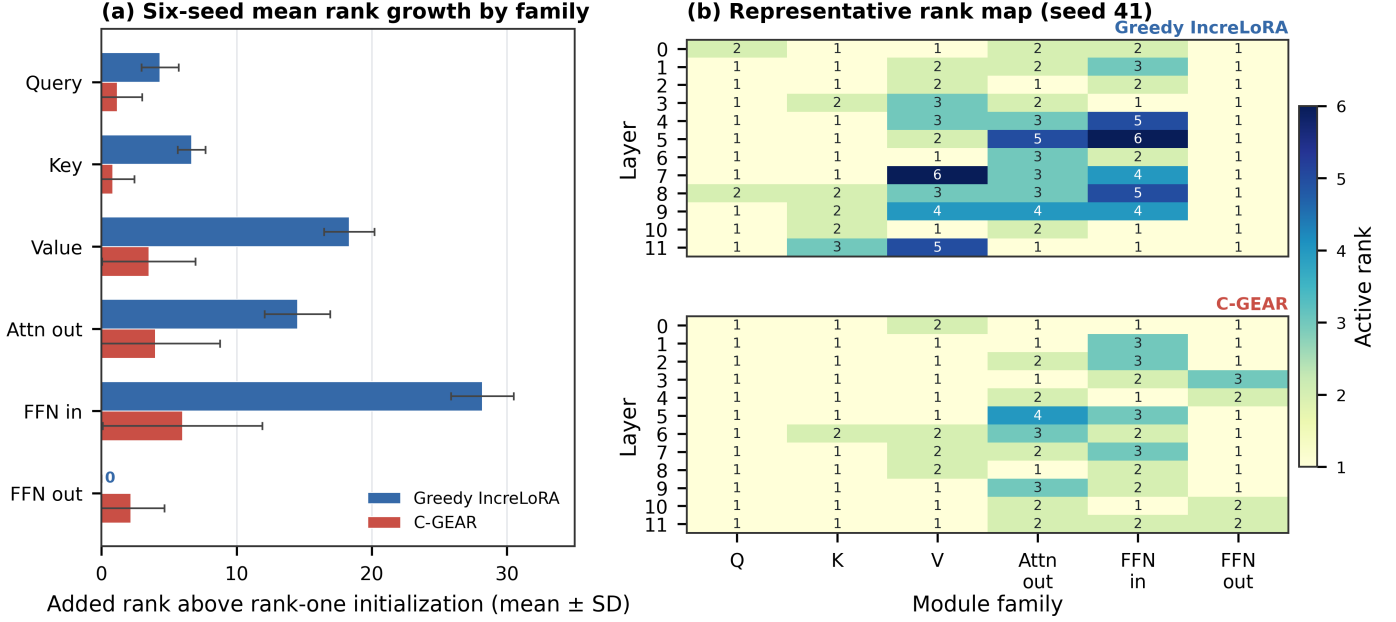


Figure 5. Allocation structure. (a) Added rank above the common rank-one initialization, summed by family per seed and shown as mean $\pm$ sample SD over seeds 41–46. (b) Final layer $\times$ module-family maps for representative seed 41 under a shared active-rank scale. C-GEAR changes both the amount and placement of rank.

window. Optimization continues with the resulting fixed pattern after allocation stops.

Evolutionary exploration also affects committed allocations: global-GA candidates supply 13 of the 40 positive-growth decisions rather than every event reverting to the Greedy anchor. For representative seed 41, training-only calibration evaluates 105 shortlisted candidates over 15 events, selects global-GA candidates three times, and selects $k = 0$ twice. Its full calibration history remains available as an auxiliary figure. Budget feasibility is enforced for every candidate; the representative seed-41 trace does not itself exhibit budget pressure.

Figure 5 shows that the smaller C-GEAR architectures are not proportional copies of Greedy. At seed 41, the final maps differ in 42 of 72 layer–family cells; C-GEAR assigns more rank in 12 cells despite using total rank 107 instead of 144. The four C-GEAR runs that grow beyond initialization develop non-uniform maps, whereas seeds 42 and 46 retain rank one throughout. Thus telemetry demonstrates changes in both *how much* rank is allocated and, when growth occurs, *where*; it does not establish that a particular pattern causally improves accuracy. Calibration and search add a measured 13.4% mean wall-time overhead in this local setup.

6 Future Work & Conclusion

The evidence is deliberately narrow: one GLUE task, one backbone, one laptop GPU, and six seeds. It does not establish statistical significance or cross-task generalization. Broader GLUE and model evaluation, larger seed sets, calibration/search ablations, and hardware-aware latency or memory objectives are natural next steps. Within this controlled study, C-GEAR supplies both a methodological improvement and a substantial implementation: it converts compulsory local Greedy growth into calibrated, budget-aware evolutionary decisions. Across six matched RTE seeds, it improves mean accuracy while using fewer selected-checkpoint parameters, and its final architectures grow substantially less.

References

- [1] N. Ding, X. Lv, Q. Wang, Y. Chen, B. Zhou, Z. Liu, and M. Sun. Sparse low-rank adaptation of pre-trained language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4133–4145, 2023.
- [2] P. He, J. Gao, and W. Chen. DeBERTaV3: Improving deberta using ELECTRA-style pre-training with gradient-disentangled embedding sharing. In *International Conference on Learning Representations*, 2023.
- [3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- [4] Y. Mao, K. Huang, C. Guan, G. Bao, F. Mo, and J. Xu. DoRA: Enhancing parameter-efficient fine-tuning with dynamic rank distribution. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 11662–11675, 2024.
- [5] M. Valipour, M. Rezagholizadeh, I. Kobayev, and A. Ghodsi. DyLoRA: Parameter-efficient tuning of pre-trained models using dynamic search-free low-rank adaptation. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 3274–3287, 2023.
- [6] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *International Conference on Learning Representations*, 2019.
- [7] F. Zhang, L. Li, J. Chen, Z. Jiang, B. Wang, and Y. Qian. IncreLoRA: Incremental parameter allocation method for parameter-efficient fine-tuning. *arXiv preprint arXiv:2308.12043*, 2023.
- [8] Q. Zhang, M. Chen, A. Bukharin, P. He, Y. Cheng, W. Chen, and T. Zhao. Adaptive budget allocation for parameter-efficient fine-tuning. In *International Conference on Learning Representations*, 2023.
- [9] R. Zhang, R. Qiang, S. A. Somayajula, and P. Xie. AutoLoRA: Automatically tuning matrix ranks in low-rank adaptation based on meta learning. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 5048–5060, 2024.